Tutorial Two: Particle Identification at LHC

What would you call the evil AI's that are helping the mosquitoes?

Proton-proton collision

Today we are going to look at some LHC data and train a support vector machine (SVM) to identify the events to be either a $t\bar{t}$ event or a 4-top ($t\bar{t}t\bar{t}$) event. Four top events are rare for the Standard Model, but if a beyond the Standard Model predicts heavy particles that can decay to a $t\bar{t}$ pair, the rate of the events can be enhanced. Interested readers can read about four tops in this paper.

Disclaimer about data

Disclaimer: The data presented in this tutorial are not the raw data. It has been cleaned up: A sample of raw data may look like this:

	P0	P1	P2	P3	P4	P5	P6	P7	P8
0	0;ttbar;0.00167779;17742.1;-2.37367;j	193103.0	143393.0	0.803969	-0.198431;j	778814.0	104707.0	2.695040	-3.03908;j
1	1;ttbar;0.00167779;15567.8;-2.90015;b	183399.0	105032.0	1.153790	-2.43549;j	166852.0	93020.5	1.186420	0.203206;j
2	2;ttbar;0.0334195;30647.2;2.74336;j	420301.0	135727.0	-1.792050	-0.609116;j	294268.0	78516.8	-1.994660	-2.51345;b
3	3;4top;1.93132e-07;27699.6;0.617912;j	300154.0	277869.0	-0.370231	-0.684062;j	261232.0	206121.0	0.698378	1.81692;j
4	4;ttbar;0.00167779;92205.9;1.57425;j	140760.0	75721.7	-1.225510	-2.92616;j	1650250.0	68573.0	-3.873460	-0.941662;j

In reality, it is necessary and crucial to do something with P0 and P4, etc. to put the features in a form that the machine learning algorithm can use (numerical arrays, for example). Please refer to Dr. Acquaviva's original notebook on how to handle raw data. In this tutorial, we will focus on SVM and directly use processed data instead. If you haven't downloaded the data, you should do it now. There are two files containing the features and the labels.

1. The data frame and imputing strategy

The data frame contains 77 columns and 5000 rows. Of those 77 columns, two correspond to the magnitude and azimuthal angle of the missing transverse energy vector, 75 contain five descriptors (object type and energy momentum 4-vector[1]) for up to 15 particles observed in the event.

a) Run the code up to the section (include) "Read in features and labels" to see the output. Take a look at the output for

```
features.describe()
```

to look at the summary statistics for each column. Pay attention to the overall count for each column. What could be the problem if we want to use all the features?

We will need to handle these issues.

b) One strategy is to discard events with missing data. Could this work? What is the problem with this strategy?

Hence, we need an imputing strategy. The most common strategy is simply setting those NAN as 0. This might convey the message that "nothing special is happening here." This is a good strategy if the missing values are few in comparison to the size of the data set. We are going to use this strategy.

c) For this strategy to be good, by looking at the "count" output, how many features should we use, up to how many particles?

For now, we are going to look at four output particles.

d) Check that there are only an insignificant percentage of data set missing the fourth particle.

Run up to the "Important" section, this will select the features we are going to use.

[1] The data is in a detector friendly form: transverse momentum, energy, pseudorapidity and azimuthal angle.

2. Target exploration and benchmarking

We should know how well our algorithm works by comparing it with a benchmark model. The labels have already been encoded as 0 (negative, $t\bar{t}$) or 1 (positive, 4-top) in a list named `target`.

a) Use your knowledge of an array to write a one line code to count the positive cases. What is the percentage that the instances are positive?

b) How accurate would the benchmark model be if the model simply tags everything negative? How accurate is it that the benchmark model assigns something random according to the class distribution?

   This is the basis for us to judge that we have a good performance.

c) Run our first model with a linear kernel and and a very large parameter C (that penalizes violation heavily). Use the cross_validation function with a 5-fold stratified(proportion in each fold is similar) CV. What is the accuracy? Is this a good value? (Remember to compare to our basis.) Examine your data carefully, what could be the problem? (You may want to consider looking at the other information from the `describe` function before.)

   We did not scale our features! Now, run a pipeline[2] to build our reference model. Run the section "How about scaling?".

d) What is the accuracy now? Is that an improvement?

e) Now take a look at the learning curve[3]: describe the training and testing learning curve using the following adjectives: high, low, increase, decrease, and flatten.

f) Why does the training score start high? Will adding more data help? Does our model suffer from high variance[4]? Does our model suffer from high bias[5]?

[2] So it will scale the feature and then use SVM in one go.

[3] The x-axis tells us how many percentage of the data we used as training data.

[4] Model has high variance if the model prediction is very sensitive to fluctuations in training data, like degree 10 polynomial fitting noisy quadratic data. Probably overfitting

[5] The model prediction is far from true value, probably because model is too simple, like using a line to fit quadratic data.

3. Hyperparameter Optimization

First, we would like to know whether selecting a more complicated model with a different kernel will be better.

a) Using a grid search, compare the model using the polynomial and Gaussian kernels, with three different degrees of the polynomial of your choice, for a fixed set of parameters γ and C of your choice. Run for a different choice of γ and C. We can also visualize the comparison in a table that ranks them. What do you find regarding the kernels?

b) Now we will proceed with a coarse grained grid search for Gaussian kernels. Change the parameter dictionary (you need to enter your choice of parameters here) for selected values γ and C that span a wide coverage. Then run the search (no other changes needed). Be careful of the curse of dimensionality and try to run around 100 fits (recall it is five folded) at a time.

c) Now we can look at the results. How does our top model compare with the simple linear model? Is there any improvement? Is it significant? Is there any problem with our top models from the grid search? Which model will you pick as the winner and why?

d) It seems our simple linear model is the best, now try to include more features by adding all the particles we previously dropped (the data has been cleaned up and called `features_all`.). Does this improve our model?

Classification Metrics in Imbalanced Datasets

The four outcomes of a binary classification are

- True Positive (TP): correctly predicting a positive example.
- True Negative (TN): Correctly predicting a negative example.
- False Positive (FP): Incorrectly predicting positive when it's actually negative.
- False Negative (FN): Incorrectly predicting negative when it's actually positive.

Some common metrics are

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{TP + TN + FP + FN} \tag{1}$$
$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

Recall tells us out of the interesting cases, how many our model catches.

The confusion matrix as you will see in the code uses a 2×2 table that summarizes all four outcomes. Rows correspond to true labels and columns to predicted labels.

4. Evaluation

When we evaluate the success of a machine learning algorithm, we have many metrics. For a very biased sample, we should try to use other metrics as well.

a) Let us first try recall: What is the recall for the lazy mode? What is the recall for a completely random assignment?

b) Now run our linear model (both the unscaled and scaled versions) where the scoring is changed to "recall". How did we compare with the lazy modes?

   In order to learn the details of the classification outcomes, we can look at the confusion matrix.

c) Run the code to compute the confusion matrix and visualize it.

d) Which kind of errors makes our current model not a great model?

5. Feature engineering

Let us use some knowledge from particle physics to add some more features. If one top decay produces some combinations of leptons, gluon-jets, b-jets, and generic jets. It is natural to think that 4-top will in general produce more of these than the $t\bar{t}$ events! Inspired by this thought, we introduce our first feature engineering.

a) Returning to the notebook, run the code first with

   `np.unique...`

What are the all possible products?

One new feature could be the total number of products. This is counted by the line that starts with

```
ntot=
```

another such feature could be the total number of b-jets and so on and so forth. The notebook counted all types.

b) Define some new features (two examples are given) and check `features.head()` to make sure the new features are there.

c) Rerun our linear SVM model (include any number of new features) to see whether including new features improves. Use accuracy, recall, and decision matrix. Why does including new features improve or not improve our model?

For a more interesting, physics-informed kernel, check out this paper. Open ended question (think about it in the end): What other features could we add? Explore them and share your discoveries with your peers and us!

6. Principle Component Analysis(PCA)

First we will review PCA and then apply to the LHC data.

PCA is performed on a matrix X^c where each column has mean 0. One can calculate X^c from X as

$$X^c_{ij} = X_{ij} - \frac{1}{M}\sum_{k=1}^{M} X_{kj} \tag{2}$$

The principle components $x'_1, x'_2, \ldots$ are stored in the columns of an $M \times N$ matrix

$$X' = X^c P \tag{3}$$

where P is an $N \times N$ matrix. P is determined by solving the eigenvalue problem

$$\frac{1}{M-1} X^{c,T} X^c = PDP^{-1} \tag{4}$$

where D is a diagonal matrix with non-negative entries $\lambda_1 \geq \lambda_2 \geq \ldots \geq \lambda_N \geq 0$.

Once we have the eigenvalues, we can define the so-called *explained variance ratio* r_l, which means how much of the variance in the data set X can be explained by the principal component x'_l. This ratio is defined in terms of the eigenvalues λ_l as

$$r_l = \frac{\lambda_l}{\sum_{i=1}^{N} \lambda_i} \tag{5}$$

Now we are going to use PCA to analyze the same LHC data.

a) Run the code to perform PCA according to the above steps: scale the features, shift the mean to zero, and solve the eigenvalue systems. The code's output is ordered by descending order λ with the corresponding P and X'.

b) Write a one-line code to compute the explained variance ratio. Plot the explained variance ratio. The code computes the cumulative-explained variance with function `cumsum`. Plot the cumulative-explained variance.

c) Make a scatter plot of x'_1 v.s. x'_2. You need to fill in the arrays to plot. The code provided label the points in your plot so that they are colored according to the labels. Interpret your plot. Does the first principle component do a good job in classification?

d) Now train an SVM on reduced dimensions (k=1,2,5,10,20, etc.). Compare its accuracy/recall/confusion matrix with the full-featured SVM.

e) Now repeat the analysis by adding the engineered features (ntot, njtot etc).

Which model overall performs the best?

A. Reference

This tutorial is written base on chapter 4 of Viviana Acquaviva's book "Machine Learning for Physics and Astronomy".

Support Vector Machine.